

Ejercicio Costos

EJERCICIO 1: Análisis de Código Secuencial vs. Iterativo

Determiná la familia de complejidad temporal O del siguiente método en el **peor caso**.

Justificá desglosando el costo de cada bloque.

```
public static void procesoMisterioso(int[] arreglo) {
    int n = arreglo.length;
    int a = 10;                // (Instrucción 1)
    System.out.println("Inicio"); // (Instrucción 2)

    for (int i = 0; i < n; i++) { // (Bucle 1)
        arreglo[i] = arreglo[i] * 2;
    }

    for (int j = 0; j < n; j++) { // (Bucle 2)
        if (arreglo[j] > 100) {
            System.out.println(arreglo[j]);
        }
    }
}
```

Bucle 1 : $O(n)$

Bucle 2 : $O(n), O(1) = O(n)$

EJERCICIO 2: El "Bardo" de los Ciclos Anidados

Analizá el método `validarPares` y determiná su complejidad temporal. ¿Qué sucede si el número que buscamos está en la primera posición? ¿Y si no está? Enfocate en el **peor caso**.

```
public static boolean validarPares(int[][] matriz) {
    int n = matriz.length;
    for (int i = 0; i < n; i++) { // Ciclo Externo
        for (int j = 0; j < n; j++) { // Ciclo Interno
            if (matriz[i][j] % 2 == 0) {
                System.out.println("Par hallado");
            }
        }
    }
    return false;
}
```

$O(n^2)$

El peor caso se dará cuando no se encuentre un número par en la matriz.

EJERCICIO 3: Comparación de Implementaciones (TDA Cola)

Basándote en las tablas de costos de la cátedra, compará la operación **Acolar(x)** en:

1. **ColaPU (Estática):** Donde el nuevo elemento va siempre a la posición 0 del arreglo.

Es más costoso ya que por cada elemento que se agrega se deben mover todos los anteriormente agregados adicionando un costo al proceso.

$O(n)$

2. **ColaLD (Dinámica):** Donde se utiliza un puntero al último nodo. **Pregunta:** ¿Cuál es el costo de cada una y por qué la implementación estática suele ser más "pesada" en este caso?

La implementación dinámica es la menos costosa posible dentro de los casos de cola ya que solo se guarda una posición para el puntero pero no requiere que se muevan todos los elementos cada vez que se agrega uno nuevo.

$O(1)$ constante